DEEP LEARNING

Interview & MNC-Project Master Guide

ANN → CNN → RNN/LSTM → NLP → Transformer → LLM

Every concept marked here is something you MUST know cold for AI/ML Engineer interviews at MNCs and for building real production-grade deep learning projects.

Prepared for: Ayan's Deep Learning Journey

github.com/aryantyagi2211/DeepLearning_Journey

1 | ARTIFICIAL NEURAL NETWORKS (ANN)

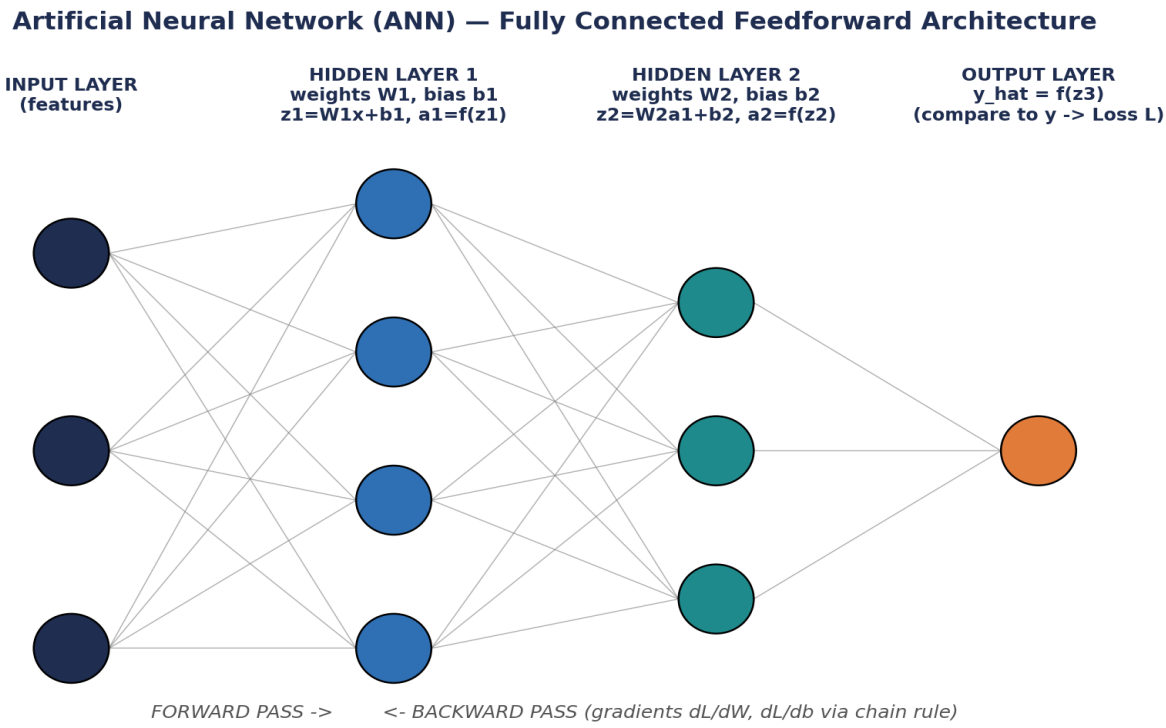

Fig 1.1 — Fully connected feedforward network: input → hidden layers → output

Core Building Blocks

Neuron / Perceptron

- A neuron computes **z = W·x + b**, then applies an activation: **a = f(z)**.
- Single perceptron can only solve **linearly separable** problems (classic interview trap: XOR problem).
- Stacking layers with non-linear activations lets the network approximate **any function** (Universal Approximation Theorem).

Activation Functions — know when to use which

- **Sigmoid**: squashes to (0,1). Used in output for binary classification. Suffers from vanishing gradient.
- **Tanh**: squashes to (-1,1), zero-centered — better than sigmoid for hidden layers, still vanishes for deep nets.
- **ReLU**: f(z)=max(0,z). Default choice for hidden layers — fast, no vanishing gradient for z>0. Risk: **Dying ReLU** (neuron stuck outputting 0 forever).
- **Leaky ReLU / PReLU**: fixes dying ReLU by allowing a small negative slope.
- **Softmax**: converts logits into a probability distribution — used in multi-class output layer.
- **GELU / SwiGLU**: smoother than ReLU, used in modern Transformers/LLMs (BERT, GPT use GELU; Llama uses SwiGLU).

Training Mechanics

Loss Functions

- **MSE (Mean Squared Error)** — regression tasks.
- **Binary Cross-Entropy** — binary classification.
- **Categorical Cross-Entropy** — multi-class classification (paired with softmax output).

Backpropagation

- Uses the **chain rule** to compute gradient of loss w.r.t. every weight: dL/dW.
- Forward pass computes predictions and loss; backward pass computes gradients layer by layer (from output back to input).
- Each layer's gradient depends on the gradient of the layer **after** it — this is why depth causes vanishing/exploding gradients.

Optimizers

- **SGD** — basic, noisy updates, needs learning-rate tuning.
- **SGD + Momentum** — smooths updates using past gradients, avoids zig-zagging.
- **RMSProp** — adapts learning rate per parameter using a moving average of squared gradients.
- **Adam** — combines Momentum + RMSProp; default choice in almost every real project.
- **AdamW** — Adam with correctly decoupled weight decay; standard for training Transformers/LLMs.

> **NOTE — Interview Trap**
>
> "Why not always use a huge learning rate?" — Too high → loss diverges/oscillates. Too low → training is painfully slow and can get stuck in a bad local minimum/saddle point. This is why **LR schedulers** (step decay, cosine decay, warmup) are used in real projects.

Stabilizing & Regularizing Deep Networks

Vanishing / Exploding Gradients

- In deep nets, gradients are a **product of many small/large numbers** through the chain rule → they shrink to ~0 (vanish) or blow up (explode).
- Fixes: ReLU family activations, **Batch/Layer Normalization**, residual/skip connections, gradient clipping, good weight initialization.

Weight Initialization

- **Xavier/Glorot init** — good for sigmoid/tanh (keeps variance stable across layers).
- **He init** — designed for ReLU (accounts for ReLU killing half the activations).
- Never initialize all weights to 0 — every neuron would learn the same thing (symmetry problem).

Normalization

- **Batch Normalization**: normalizes activations across the **batch** dimension; speeds up training, adds slight regularization. Behaves differently in train vs eval mode (interview favorite).
- **Layer Normalization**: normalizes across the **feature** dimension per sample — works with batch size 1, used everywhere in Transformers/LLMs.

Regularization (prevents overfitting)

- **Dropout**: randomly zeroes out neurons during training so the network can't over-rely on specific paths.

- **L1/L2 (weight decay)**: penalizes large weights; L1 encourages sparsity, L2 encourages small smooth weights.
- **Early stopping**: stop training when validation loss stops improving.
- **Data augmentation**: artificially grows the dataset (mainly used in CNN/vision).

> **NOTE — Must Remember for Projects**
>
> Always track **train loss vs validation loss**. Train↓ Val↑ = overfitting (add regularization/more data). Both high = underfitting (bigger model / train longer / better features).

1.X — Quick Recall: ANN

Q1. Why can't a single perceptron solve XOR?

A: XOR is not linearly separable — a single decision boundary (line) can't split the classes. Needs a hidden layer to create a non-linear boundary.

Q2. Why does ReLU generally train faster than sigmoid in deep nets?

A: ReLU's gradient is either 0 or 1 (no squashing for positive inputs), so it doesn't shrink gradients across many layers the way sigmoid's saturating curve does.

Q3. What is the difference between Batch Norm and Layer Norm?

A: BatchNorm normalizes across the batch dimension (needs batch size > 1, stats differ train/eval). LayerNorm normalizes across features per single sample (works for batch size 1, used in Transformers).

Q4. Adam vs SGD — when would you still pick SGD?

A: SGD (with momentum) sometimes generalizes better and is preferred for very large-scale vision training (e.g. ResNet on ImageNet) where careful LR schedules are tuned. Adam converges faster and is the safe default elsewhere.

Q5. What problem does Dropout solve and how does the code differ at test time?

A: Prevents co-adaptation/overfitting by randomly disabling neurons in training. At test time dropout is turned OFF and outputs are scaled to keep expected activation magnitude the same.

2 | CONVOLUTIONAL NEURAL NETWORKS (CNN)

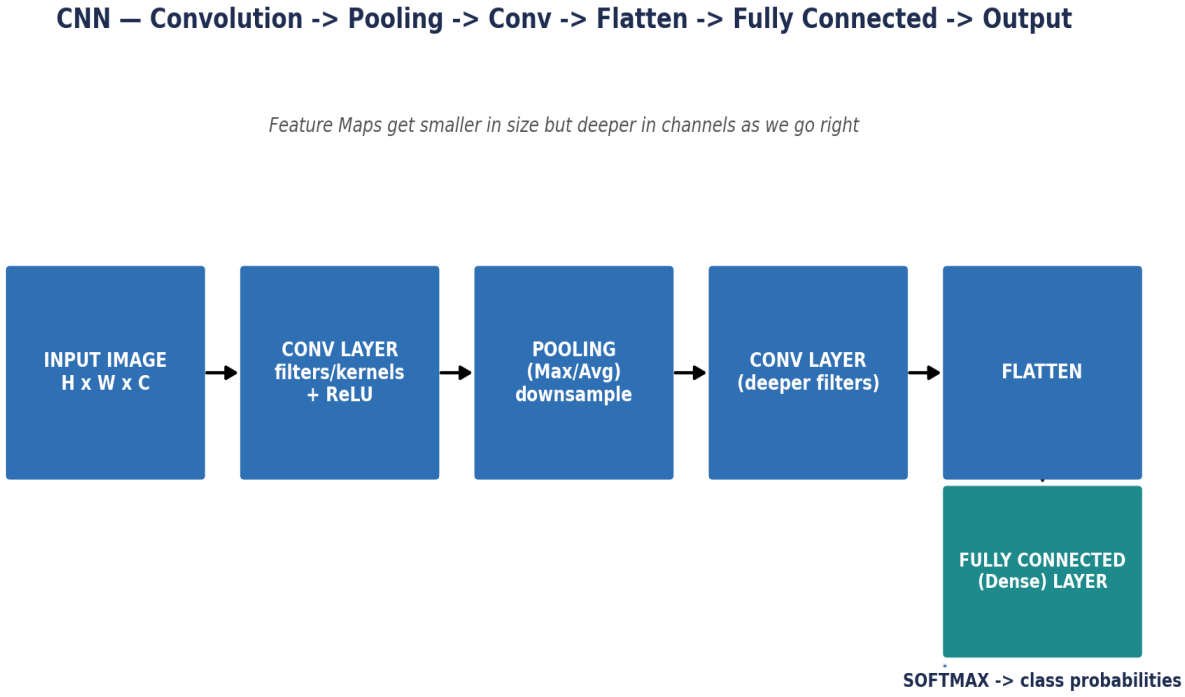

Fig 2.1 — Classic CNN pipeline: Conv → Pool → Conv → Flatten → Dense → Softmax

Core Building Blocks

Convolution Layer

- A small **filter/kernel** slides over the image computing a dot product — detects local patterns (edges, textures, shapes).
- Early layers learn simple features (edges); deeper layers learn complex features (shapes, object parts, whole objects).
- **Parameter sharing** (same filter reused everywhere) is why CNNs have far fewer parameters than a fully connected net on images.
- Output size formula: **Output = (Input** – **Kernel + 2×Padding) / Stride + 1**.

Padding & Stride

- **Padding** = adding zeros around the border. 'Same' padding keeps output size = input size; 'Valid' padding = no padding, output shrinks.
- **Stride** = step size the filter moves. Stride 2 halves spatial size — used instead of pooling in some modern architectures.

Pooling

- **Max Pooling** — takes the maximum value in a window; keeps strongest activation, adds translation invariance.
- **Average Pooling** — takes the average; smoother, less commonly used in hidden layers, common in **Global Average Pooling** before final classification.
- Pooling has **no learnable parameters** — purely a downsampling operation.

Receptive Field

- The region of the original image that a single neuron in a deep layer 'sees'. Grows with depth, stride, and kernel size.
- Interview favorite: **why stack small 3x3 filters instead of one big 7x7?** → Same receptive field, fewer parameters, more non-linearities (more ReLUs) → better feature learning (this is the core idea behind VGGNet).

> **NOTE — Interview Trap**
>
> Convolution is technically **cross-correlation** in most DL frameworks (no kernel flipping like true mathematical convolution) — but everyone still calls it 'convolution'.

Modern Architecture Ideas (asked in every MNC interview)

Key Architecture Concepts

- **1x1 Convolution**: changes channel depth without touching spatial size — used for dimensionality reduction (e.g. Inception, ResNet bottleneck blocks).
- **Residual Connections (ResNet)**: adds input back to output of a block (x + F(x)) — solves vanishing gradient in very deep networks, allows training 100+ layer networks.
- **Batch Normalization** after conv layers stabilizes and speeds up training (same concept as ANN section, applied per-channel here).
- **Transfer Learning**: reuse a model pretrained on a huge dataset (ImageNet) and fine-tune on your small dataset — massively outperforms training from scratch when data is limited.

> **NOTE — Empirical Result Worth Remembering**
>
> Transfer learning vs training from scratch on the same small dataset: ~**75% accuracy (transfer learning) vs ~30% (from scratch)**. Always prefer a pretrained backbone unless you have a huge labeled dataset.

Object Detection — high-value interview topic

Two-Stage Detectors

- **R-CNN**: Selective Search proposes ~2000 regions on the **raw image** (not CNN output) → each region warped and passed through CNN separately → very slow.
- **Fast R-CNN**: runs CNN once on full image, then uses **ROI Pooling** to extract fixed-size features for each proposed region from the shared feature map — much faster than R-CNN.
- **Faster R-CNN**: replaces Selective Search with a learnable **Region Proposal Network (RPN)** — end-to-end trainable, real interview favorite for 'evolution of R-CNN family'.

Single-Stage Detectors

- **SSD (Single Shot Detector)**: predicts boxes + classes directly from multiple feature map scales in one pass — faster than two-stage, slightly less accurate on small objects.
- **YOLO (You Only Look Once, e.g. YOLOv8)**: divides image into a grid, each cell predicts bounding boxes + class probabilities directly — real-time speed, industry-standard for production object detection.

Core Object Detection Concepts

- **Anchor Boxes**: predefined boxes of different scales/ratios used as references for predicting real object boxes.
- **IoU (Intersection over Union)**: overlap between predicted box and ground truth = Area of Overlap / Area of Union. Used to judge if a prediction is 'correct'.
- **Non-Max Suppression (NMS)**: removes duplicate overlapping boxes for the same object, keeping only the highest-confidence one.
- **mAP (mean Average Precision)**: the standard metric to compare object detection models.

> **NOTE — Common Mix-up to Avoid**
>
> **ROI Pooling ≠ image warping.** ROI Pooling extracts a fixed-size feature-map crop for each region proposal directly from the shared conv feature map — it does NOT resize/warp the raw image like the original R-CNN did.

2.X — Quick Recall: CNN

Q1. Why do modern CNNs stack multiple 3x3 filters instead of one large filter?

A: Same receptive field as a larger filter, but fewer parameters and more non-linear activations in between, which improves representational power.

Q2. What problem do residual connections solve?

A: Vanishing gradients in very deep networks — the skip connection gives gradients a direct path back, allowing 100+ layer networks to train.

Q3. Difference between R-CNN and Fast R-CNN?

A: R-CNN runs the CNN separately on every region proposal (slow). Fast R-CNN runs the CNN once on the whole image and uses ROI Pooling to extract per-region features from the shared feature map (much faster).

Q4. What does IoU measure and what is it used for?

A: Overlap between predicted and ground-truth boxes (Intersection/Union). Used to decide true positives during evaluation and to filter boxes in NMS.

Q5. Why is transfer learning almost always preferred over training from scratch?

A: Pretrained models already learned general low/mid-level visual features (edges, textures) from huge datasets; fine-tuning on a small dataset reuses this and avoids overfitting, giving much higher accuracy with less data.

3 | RNN, LSTM & GRU (SEQUENCE MODELS)

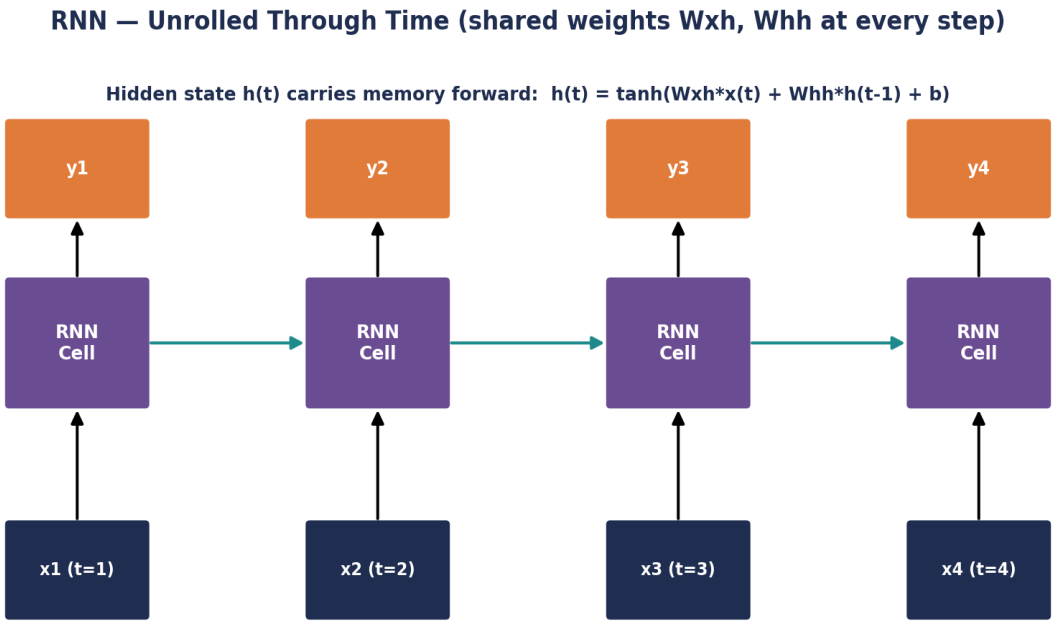

Fig 3.1 — RNN unrolled through time; same weights reused at every timestep

Why Sequence Models Exist

Core Idea

- ANN/CNN assume inputs are independent — bad for data where **order matters** (text, time series, audio, video frames).
- RNN keeps a **hidden state h(t)** that carries information from previous timesteps forward — this is its 'memory'.
- Same weight matrices (Wxh, Whh, Why) are reused at every timestep — this is **parameter sharing across time**.

The Core RNN Problem — Vanishing/Exploding Gradients Through Time

- Backprop Through Time (BPTT) multiplies gradients across every timestep — for long sequences this product shrinks to ~0 (vanishing) or blows up (exploding).
- Result: vanilla RNNs **forget long-range dependencies** — can't connect information from 50 steps ago to the current prediction.
- This single limitation is *why LSTM/GRU were invented* — guaranteed interview question.

LSTM — The Fix

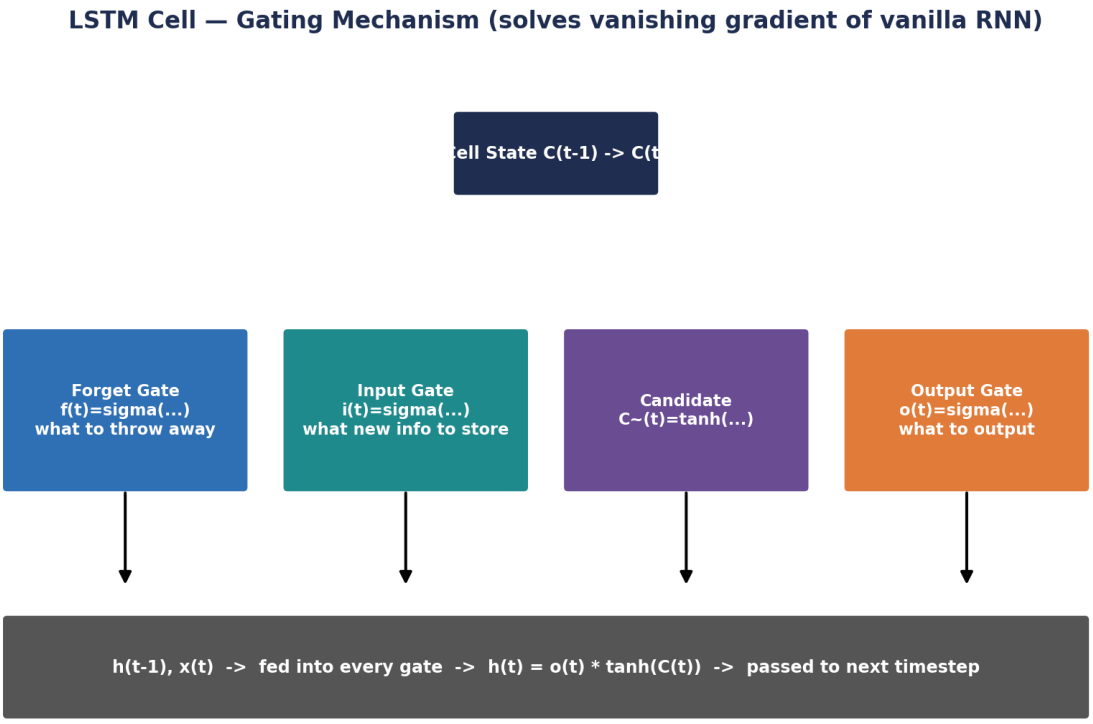

Fig 3.2 — LSTM cell: Forget, Input, Candidate, and Output gates control the cell state

LSTM Gates (know each one's job)

- **Forget Gate f(t)**: sigmoid decides what fraction of the old cell state C(t-1) to keep vs discard.
- **Input Gate i(t)**: sigmoid decides how much new information to let in.
- **Candidate C~(t)**: tanh creates the new candidate values to potentially add to the cell state.
- **Cell State Update**: C(t) = f(t)*C(t-1) + i(t)*C~(t) — this **additive** update (not repeated multiplication) is exactly what prevents vanishing gradients.
- **Output Gate o(t)**: sigmoid decides what part of the cell state becomes the hidden state h(t) = o(t)*tanh(C(t)).

GRU (Gated Recurrent Unit) — the lighter alternative

- Merges forget + input gates into a single **Update Gate**, and adds a **Reset Gate**.
- No separate cell state — just the hidden state. Fewer parameters than LSTM, trains faster, often similar performance.
- Interview one-liner: 'GRU is a simplified LSTM — fewer gates, no separate cell state, faster to train.'

Architectures Built on RNN/LSTM

Sequence Modeling Patterns

- **Seq2Seq (Encoder-Decoder)**: encoder RNN compresses input sequence into a fixed context vector; decoder RNN generates output sequence from it. Used for translation, summarization before Transformers took over.
- **Bidirectional RNN/LSTM**: processes sequence forward AND backward, concatenates both — sees full context (past + future) at each step. Can't be used for real-time/streaming generation.
- **Attention (early form, pre-Transformer)**: solved the Seq2Seq bottleneck of squeezing the whole input into ONE fixed vector — let the decoder 'look back' at all encoder states, weighted by relevance. This idea directly led to the Transformer.

NOTE — Interview Trap

"Why did Transformers replace RNNs for most NLP tasks?" — RNNs process tokens **sequentially** (step t needs step t-1), so they can't be parallelized during training. Transformers process all tokens **in parallel** via self-attention, making them far faster to train on modern GPUs, plus they capture long-range dependencies better.

3.X — Quick Recall: RNN / LSTM

Q1. Why do vanilla RNNs struggle with long sequences?

A: Backprop through time multiplies gradients repeatedly across timesteps, causing them to vanish (or explode) — the network can't learn dependencies that span many steps.

Q2. What is the key architectural trick that lets LSTM avoid vanishing gradients?

A: The cell state update is additive (C(t) = f*C(t-1) + i*C~(t)), not purely multiplicative — gradients can flow through this additive path largely unchanged across many timesteps.

Q3. GRU vs LSTM — main structural difference?

A: GRU has no separate cell state and combines the forget/input gates into a single update gate plus a reset gate — fewer parameters, simpler, usually faster to train.

Q4. What problem does attention solve in Seq2Seq models?

A: It removes the bottleneck of compressing the entire input sequence into one fixed-length context vector by letting the decoder attend to all encoder hidden states, weighted by relevance, at every decoding step.

4 | NATURAL LANGUAGE PROCESSING (NLP) FOUNDATIONS

Text Representation — from counting to meaning

Classical Methods (still asked in interviews)

- **Bag of Words (BoW)**: counts word occurrences, ignores order and grammar — simple baseline, loses context.
- **TF-IDF**: weighs a word by how often it appears in a document (TF) vs how common it is across all documents (IDF) — down-weights generic words like 'the', up-weights distinctive words.
- **N-grams**: captures short local word order (bigrams, trigrams) that BoW misses.

Text Preprocessing Pipeline

- **Tokenization**: splitting text into words/sub-words/characters.
- **Stemming**: crude chopping of word endings ('running' → 'runn') — fast but can produce non-words.
- **Lemmatization**: proper dictionary-based reduction ('running' → 'run') — slower, more accurate.
- **Stop-word removal**: dropping very common low-information words ('is', 'the', 'a').
- **POS Tagging**: labeling each word's grammatical role (noun, verb, adjective) — useful for parsing, NER pipelines.
- **Named Entity Recognition (NER)**: identifying names of people, organizations, locations, dates in text.

Word Embeddings — the shift from counting to meaning

Dense Vector Representations

- **One-Hot Encoding**: each word is a sparse vector with a single 1 — huge dimensionality, no notion of similarity between words.
- **Word2Vec**: learns dense vectors where similar words end up close in vector space. Two training strategies: **CBOW** (predict word from context) and **Skip-gram** (predict context from word).
- **GloVe**: builds embeddings from global word co-occurrence statistics across the whole corpus (vs Word2Vec's local context window).
- Famous property: vector('King') − vector('Man') + vector('Woman') ≈ vector('Queen') — embeddings capture semantic relationships as directions in space.
- **Limitation of Word2Vec/GloVe**: **static** embeddings — the word 'bank' has the same vector whether it means river bank or money bank. This is exactly the gap that **contextual embeddings** (BERT, Transformers) fill.

> **NOTE — Interview Trap**
>
> "What's the key limitation of Word2Vec that Transformers solve?" — Word2Vec gives one fixed vector per word regardless of context. Transformer-based models generate a **different embedding for the same word depending on surrounding context** (contextual embeddings).

Sequence Labeling & Language Modeling Basics

Core Tasks

- **Language Modeling**: predicting the next word given previous words — the fundamental pretraining objective behind GPT-style LLMs.
- **Text Classification**: sentiment analysis, spam detection, topic classification — usually built on top of embeddings + a classifier head.
- **Sequence-to-Sequence tasks**: translation, summarization — map an input sequence to an output sequence of possibly different length.

4.X — Quick Recall: NLP Foundations

Q1. Why is TF-IDF usually better than plain Bag-of-Words?

A: TF-IDF down-weights very common words (like 'the') that appear in almost every document and up-weights rarer, more distinctive words — giving more informative features.

Q2. Stemming vs Lemmatization — key difference?

A: Stemming crudely chops word endings using rules (can produce non-words); Lemmatization uses vocabulary + grammar rules to return the correct dictionary base form.

Q3. What is the biggest limitation of Word2Vec/GloVe embeddings?

A: They are static — one fixed vector per word regardless of context, so they can't distinguish different meanings of the same word (e.g. 'bank').

Q4. What does CBOW predict, and what does Skip-gram predict?

A: CBOW predicts the target word from its surrounding context words; Skip-gram does the reverse — predicts the surrounding context words from the target word.

5 | THE TRANSFORMER ARCHITECTURE

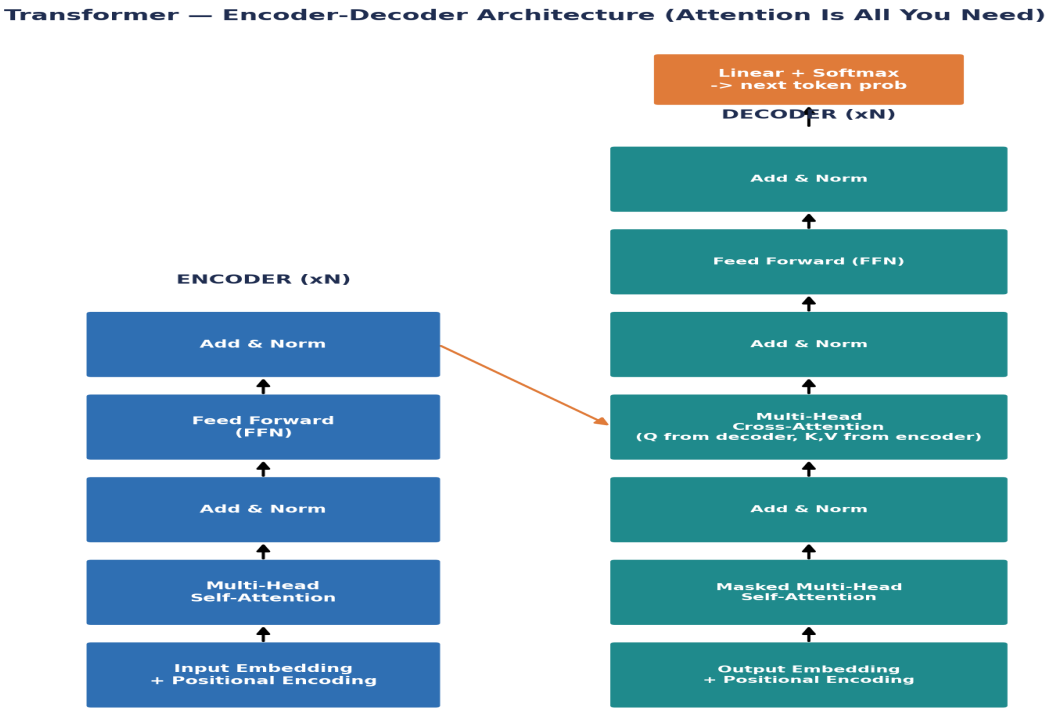

Fig 5.1 — Full Encoder-Decoder Transformer ("Attention Is All You Need", 2017)

Self-Attention — the core mechanism

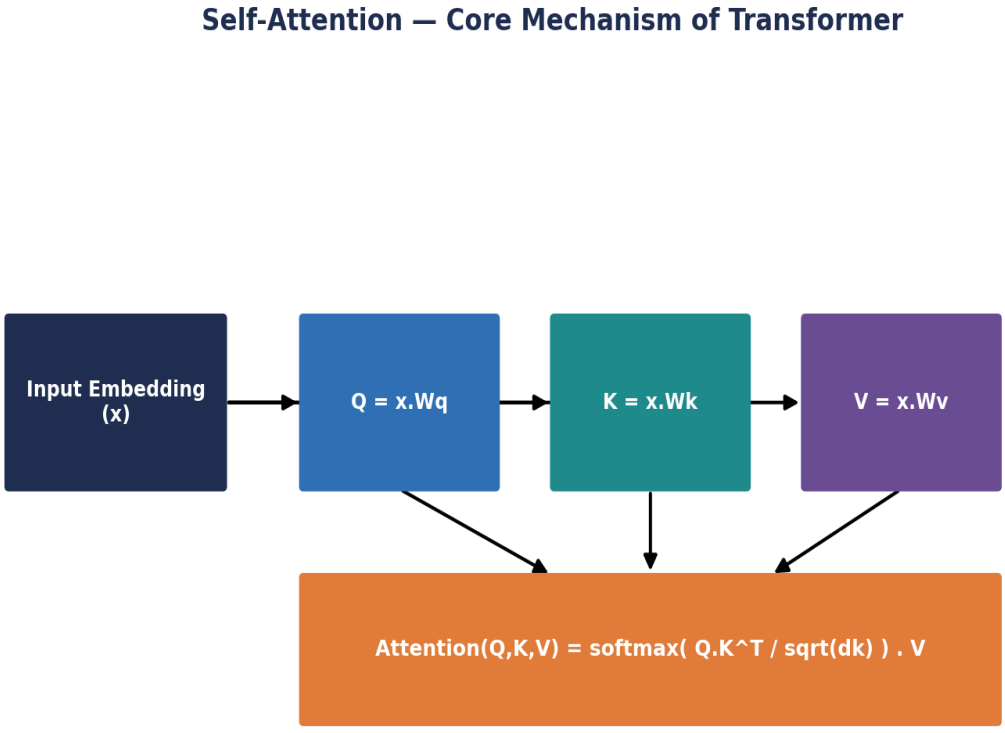

Fig 5.2 — Scaled Dot-Product Attention: Query, Key, Value

Query, Key, Value

- Every input token is projected into three vectors: **Query (Q)**, **Key (K)**, **Value (V)** using learned weight matrices.
- Attention score = how much a token should 'attend to' every other token = Q·K^T (dot product similarity).
- Scores are scaled by 1/sqrt(dk) to keep gradients stable, then passed through **softmax** to get attention weights (sum to 1).
- Final output = weighted sum of Value vectors using these attention weights: **Attention(Q,K,V) = softmax(QK^T/√dk)·V**.

Multi-Head Attention

- Instead of one attention computation, the model runs **several attention 'heads' in parallel**, each with its own Q/K/V projections.
- Each head can learn to focus on different relationships (e.g. one head tracks syntax, another tracks long-range coreference).
- Outputs of all heads are concatenated and linearly projected back to the model dimension.

Masking

- **Padding mask**: ignores padding tokens added to make sequences equal length in a batch.
- **Causal/Look-ahead mask**: used in the decoder — prevents a token from attending to **future** tokens, so predictions only depend on past context. This is exactly what makes GPT-style generation autoregressive.

Cross-Attention (Encoder-Decoder models only)

- In the decoder, Queries come from the **decoder**, but Keys and Values come from the **encoder output**.
- This is how the decoder 'looks at' the full input sequence while generating each output token (used in translation-style Transformers, not in GPT-style decoder-only LLMs).

Why Transformers Won

Key Architectural Advantages

- **Parallelization**: unlike RNNs, all tokens are processed simultaneously — massively faster training on GPUs.
- **Positional Encoding**: since attention has no built-in notion of order, sinusoidal (or later, rotary) position signals are added/injected so the model knows token order.
- **Residual connections + LayerNorm** around every sub-block — same purpose as in ANN/CNN: stabilize training of deep stacks.
- **Feed-Forward Network (FFN)** after attention in each block — adds per-token non-linear transformation capacity.

Encoder-only vs Decoder-only vs Encoder-Decoder

- **Encoder-only (BERT)**: sees the full sentence at once (bidirectional) — great for understanding tasks (classification, NER, embeddings).
- **Decoder-only (GPT, Llama)**: causal masking, generates text one token at a time — the architecture behind almost all modern LLMs.
- **Encoder-Decoder (T5, original Transformer)**: best for sequence-to-sequence tasks like translation/summarization.

NOTE — Interview Trap

"Why divide by √dk in attention?" — Without scaling, dot products grow large in magnitude as dimension increases, pushing softmax into regions with tiny gradients (saturated). Scaling keeps the softmax input in a well-behaved range for stable gradients.

5.X — Quick Recall: Transformer

Q1. What is the formula for scaled dot-product attention?

A: Attention(Q,K,V) = softmax(QK^T / sqrt(dk)) · V — dk is the dimension of the key vectors, used to scale down the dot products before softmax.

Q2. Why use multiple attention heads instead of one big one?

A: Different heads can specialize in different types of relationships (short-range syntax, long-range dependencies, etc.), giving the model richer representational power at roughly the same parameter budget.

Q3. What does the causal mask in a decoder do, and why is it needed?

A: It blocks each position from attending to future tokens, so the model can only use past context — required so training (teacher forcing) matches real autoregressive generation at inference.

Q4. Encoder-only vs Decoder-only — when would you pick each?

A: Encoder-only (BERT) for understanding/embedding tasks needing full bidirectional context. Decoder-only (GPT/Llama) for open-ended text generation where output is produced one token at a time.

6 | LARGE LANGUAGE MODELS (LLM)

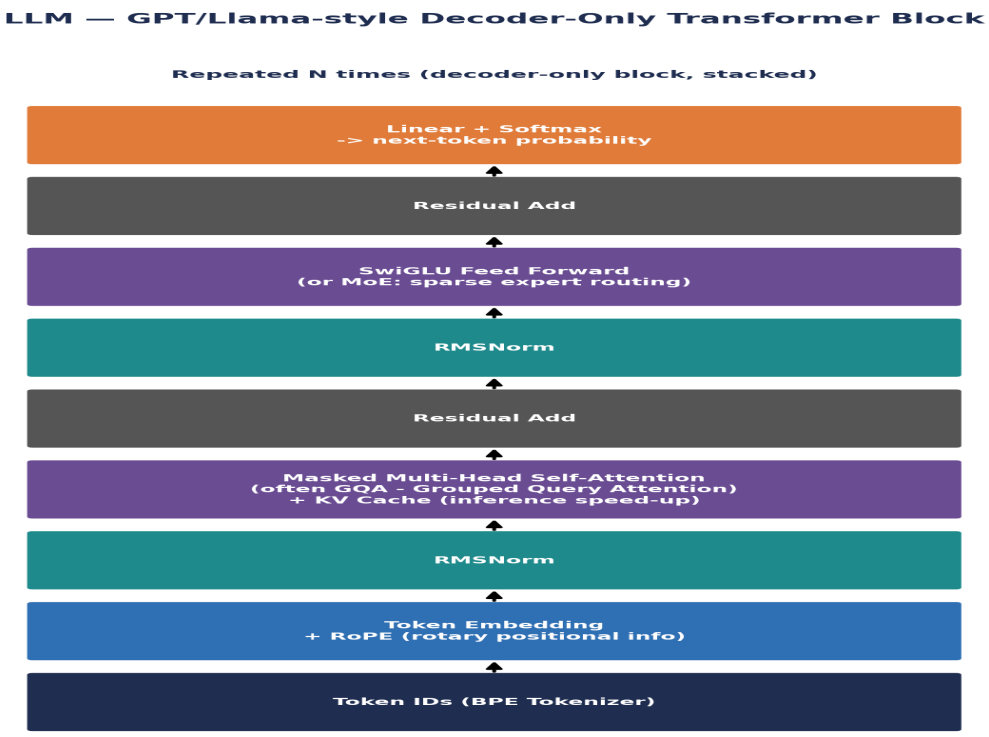

Fig 6.1 — GPT/Llama-style decoder-only block, stacked N times

Tokenization

Byte Pair Encoding (BPE) & friends

- LLMs don't see words — they see **tokens** (sub-word units). Rare words get split into smaller known pieces; common words stay whole.
- **BPE**: starts from characters, iteratively merges the most frequent adjacent pair into a new token, builds a fixed-size vocabulary (e.g. GPT uses ~50k-100k tokens).
- Why sub-words, not full words or characters: full-word vocab explodes in size and can't handle unseen words; character-level is too long/inefficient. Sub-word is the sweet spot.

Embeddings & Positional Information

From Token to Vector

- Each token ID is looked up in an **embedding table** to get a dense vector — this table is learned during training.
- **RoPE (Rotary Positional Embedding)**: instead of adding a separate position vector (like original Transformer), RoPE **rotates** the Query/Key vectors by an angle depending on position — encodes relative position directly inside the attention computation. Used in Llama, GPT-NeoX and most modern LLMs.
- Why RoPE won over learned/sinusoidal absolute position embeddings: it generalizes better to sequence lengths longer than seen in training, and naturally encodes **relative** distance between tokens (which matters more than absolute position for language).

Modern Architecture Variants (interview-critical for 2025-26 roles)

Normalization & Activation Upgrades

- **RMSNorm** (replaces LayerNorm in Llama/modern LLMs): normalizes using only the root-mean-square of activations (no mean-centering) — simpler, faster, empirically works just as well.
- **SwiGLU** (replaces plain ReLU/GELU FFN): a gated activation (Swish combined with a GLU-style gate) used in the feed-forward block — improves quality for roughly the same compute cost.

Attention Variants — Efficiency at Scale

- **MHA (Multi-Head Attention)** — original: separate K,V for every head — most expressive, most memory-hungry.
- **MQA (Multi-Query Attention)** — all heads share a **single** K,V — much less memory, some quality loss.
- **GQA (Grouped-Query Attention)** — middle ground: heads are split into groups, each group shares one K,V — used in Llama 2/3 and most modern LLMs. Big memory saver for **KV cache** at inference with minimal quality loss.

> **NOTE — Interview Trap**
>
> "Why does GQA matter for production?" — At inference, the KV cache (stored Keys/Values for every previous token) is a major GPU memory bottleneck for long conversations. GQA shrinks this cache size by sharing K,V across grouped heads, letting you serve longer contexts / bigger batches on the same hardware.

BERT vs GPT — a classic comparison question

Key Contrast

- **BERT**: encoder-only, bidirectional context, trained with **Masked Language Modeling** (predict randomly masked tokens using both left+right context) + Next Sentence Prediction. Best for understanding/embedding tasks.
- **GPT**: decoder-only, causal (left-to-right) context, trained with plain **next-token prediction**. Best for generation.
- Because BERT sees both directions, it CANNOT be used to naturally generate text left-to-right — that's why GPT-style models dominate the LLM/chatbot space.

Pretraining, Scaling & Inference

Pretraining

- Objective: **next-token prediction** on massive raw text corpora (self-supervised — no manual labels needed).
- After pretraining: **Fine-tuning / Instruction-tuning** teaches the model to follow instructions; **RLHF/DPO** aligns it with human preferences — this is what turns a raw base model into an assistant like ChatGPT/Claude.

Scaling Laws

- Model performance improves predictably as you scale **model size, dataset size, and compute** together (Kaplan/Chinchilla scaling laws).

- **Chinchilla finding**: many early LLMs were **over-parameterized and under-trained** — for a fixed compute budget, better to use a smaller model trained on more tokens than a huge model trained on too little data.

Inference Optimization

- **KV Cache**: during generation, Key/Value vectors for already-processed tokens are cached and reused instead of recomputed every step — turns generation from $O(n^2)$ recompute into an efficient incremental process. This is THE reason LLM inference is fast enough for chat.
- **Context Length**: the maximum number of tokens the model can attend to at once; extending it is a major engineering challenge (memory grows with KV cache size, and models need special training/position-encoding tricks to generalize to longer contexts than seen in training).
- **MoE (Mixture of Experts)**: instead of one dense FFN, the model has many 'expert' FFN sub-networks and a **router** picks a small subset (e.g. 2 of 8) to activate per token — massively increases total parameters without proportionally increasing compute per token. Used in Mixtral, GPT-4 (rumored), and other frontier models.

> **NOTE — Must Remember for Projects**
>
> MoE gives you a **large model's knowledge capacity** at a **small model's inference compute cost** — because only a few experts fire per token. The trade-off is more total GPU memory needed to hold all experts, and added routing complexity/training instability.

6.X — Quick Recall: LLM

Q1. Why do LLMs use sub-word tokenization (BPE) instead of whole-word tokens?

A: Whole-word vocabularies explode in size and can't handle unseen/rare words. BPE keeps common words whole and splits rare words into known sub-word pieces, giving a compact, flexible vocabulary.

Q2. What advantage does RoPE have over standard learned positional embeddings?

A: RoPE encodes relative position directly into the attention dot product via rotation, which generalizes better to sequence lengths beyond what was seen in training and captures relative distance more naturally.

Q3. Explain GQA in one sentence and why it matters at inference time.

A: Grouped-Query Attention shares Key/Value projections across groups of attention heads, shrinking the KV cache size at inference and reducing GPU memory pressure with minimal quality loss versus full multi-head attention.

Q4. What is the Chinchilla scaling law finding, in simple terms?

A: For a fixed compute budget, it's better to train a smaller model on more data (tokens) than to train a much larger model on too little data — many early LLMs were undertrained relative to their size.

Q5. How does the KV cache make autoregressive generation efficient?

A: It stores the Key/Value vectors computed for every previously generated token so they don't need to be recomputed at each new step — the model only computes Q/K/V for the newest token and reuses the cache for everything before it.

Q6. What problem does Mixture-of-Experts (MoE) solve?

A: It lets a model have very high total parameter count (knowledge capacity) while keeping the compute cost per token low, by only activating a small subset of 'expert' sub-networks per token via a learned router.

7 | RAPID-FIRE INTERVIEW CHEAT SHEET

One-line answers for the questions that come up again and again in ML/AI Engineer interviews. Use this as a final revision pass before an interview.

Term	One-line definition / interview answer
Vanishing gradient	Gradients shrink toward 0 across many layers/timesteps → early layers stop learning. Fixed by ReLU, residuals, norm layers, LSTM gating.
Overfitting	Model memorizes training data, fails to generalize. Fix: more data, dropout, regularization, early stopping.
Bias-Variance tradeoff	High bias = underfitting (too simple). High variance = overfitting (too complex, sensitive to training data).
Batch Norm vs Layer Norm	BatchNorm: across batch dim, needs batch>1. LayerNorm: across feature dim per sample, works in Transformers.
Receptive field	Region of input image a neuron's output depends on; grows with depth/stride/kernel size.
IoU	Intersection over Union — overlap metric between predicted and ground-truth boxes in object detection.
NMS	Non-Max Suppression — removes duplicate overlapping detection boxes, keeps highest-confidence one.
Transfer learning	Reuse pretrained model weights and fine-tune on your (smaller) dataset instead of training from scratch.
Vanishing gradient in RNN	Repeated multiplication of small gradients across timesteps in BPTT → long-range dependencies are lost.
LSTM vs GRU	LSTM: 3 gates + separate cell state, more expressive. GRU: 2 gates, no separate cell state, fewer params, faster.
Self-Attention	Every token computes a weighted combination of all other tokens' values, weighted by Query-Key similarity.
Why scale by sqrt(dk)	Keeps dot-product magnitudes in a range where softmax gradients don't vanish.
Causal mask	Prevents a decoder position from attending to future tokens — enables autoregressive generation.
BERT vs GPT	BERT: encoder-only, bidirectional, masked-LM objective, good for understanding. GPT: decoder-only, causal, next-token objective, good for generation.
RoPE	Rotary Positional Embedding — encodes position by rotating Q/K vectors; generalizes well to longer contexts.
RMSNorm	Simplified LayerNorm using only root-mean-square (no mean subtraction) — used in Llama-style LLMs.
GQA	Grouped-Query Attention — groups of heads share K/V, shrinking KV-cache memory at inference vs full MHA.
KV Cache	Cached Key/Value vectors from previous tokens, reused at each generation step to avoid recomputation.

Term	One-line definition / interview answer
MoE	Mixture of Experts — router activates only a few expert FFN sub-networks per token, boosting capacity without proportional compute cost.
Scaling laws / Chinchilla	Performance scales predictably with model size + data + compute; for fixed compute, balance model size and training tokens (don't just make the model bigger).
Fine-tuning vs RLHF/DPO	Fine-tuning: supervised training on task/instruction data. RLHF/DPO: aligns model outputs to human preference rankings, done after fine-tuning.